JavaScript
Interview Cheat Sheet

Memory model · Object · Array · Map · Set · CRUD · all core methods

Prepared for **Suraj Khonde** — glance before you walk in

1 · How JavaScript Stores Data (Memory Model)

The #1 conceptual question. Primitives live by value; objects live by reference.

Two kinds of values

Primitives (`string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`) are **immutable** and copied **by value**. Conceptually they sit on the **stack** — small, fixed size.
Objects (objects, arrays, functions) are stored on the **heap**. The variable only holds a **reference (pointer)** to that heap location. Assignment copies the *reference*, not the data.

STACK (by value)

```
let a = 10; // 10
let b = a; // copy 10
b = 20; // a still 10
```

HEAP (by reference)

```
let o = {x:1}; // ref -> {x:1}
let p = o; // same ref
p.x = 9; // o.x is 9 too!
```

Consequences you must be able to explain

Concept	Behaviour
`Equality ==`	Primitives compared by **value**; objects compared by **reference** (same pointer).
`const obj`	You **can mutate** the contents; you **cannot reassign** the variable.
`Function args`	Primitives passed by value (copy); objects pass the **reference**, so mutations leak out.
`Shallow copy`	`{...o}`, `Object.assign({},o)`, `arr.slice()` copy **one level**; nested objects stay shared.
`Deep copy`	`structuredClone(o)` (modern) or `JSON.parse(JSON.stringify(o))` (loses funcs/dates).
`Garbage collect`	Heap memory is freed automatically when nothing references it (mark-and-sweep).

```
const a = { n: 1 };
const b = a;                // same reference
b.n = 99;
console.log(a.n);           // 99  (shared!)
console.log(a === b);       // true (same object)
console.log({n:1} === {n:1});// false (different objects)

const shallow = { ...a };   // copies top level only
const deep = structuredClone(a);
```

2 · Object — CRUD + Methods

Key-value store. Keys are strings or symbols. Best for fixed, known shapes.

CRUD (Create · Read · Update · Delete)

```
// CREATE
const user = { id: 1, name: 'Suraj' };        // literal (preferred)
const u2  = new Object();                       // rarely used
const u3  = Object.create(proto);               // with a prototype
const u4  = { ...user, role: 'admin' };         // clone + add

// READ
user.name;            // 'Suraj'  (dot)
user['name'];         // 'Suraj'  (bracket - dynamic key)
const { id, name } = user;                      // destructuring
user?.address?.city;  // optional chaining (safe)

// UPDATE
user.name = 'S';                 // mutate
Object.assign(user, { age: 30 }); // merge (mutates target)
const next = { ...user, age: 31 };// new object (immutable update)

// DELETE
delete user.age;      // removes the key
```

Object static methods

Method	Returns / does
`Object.keys(o)`	Array of own enumerable **keys** → `['id','name']`
`Object.values(o)`	Array of own **values** → `[1,'Suraj']`
`Object.entries(o)`	Array of `[key, value]` pairs (great with `for...of`)
`Object.fromEntries(p)`	Builds an object from `[k,v]` pairs (reverse of entries)
`Object.assign(t,...s)`	Copies sources into target, **mutates & returns** target
`Object.freeze(o)`	Makes object **immutable** (shallow); `isFrozen` to check
`Object.seal(o)`	No add/remove keys, but existing values **can** change
`Object.create(p)`	New object with `p` as its prototype
`Object.hasOwn(o,k)`	`true` if own (not inherited) key — modern `hasOwnProperty`
`Object.getPrototypeOf(o)`	Returns the prototype (`__proto__`)
`Object.defineProperty`	Add a key with fine control (writable/enumerable/getter)
`'k' in o`	`true` if key exists (own **or** inherited)

Iterating an object

```
for (const key in user) { /* keys, incl. inherited - use hasOwn guard */ }
for (const [k, v] of Object.entries(user)) { console.log(k, v); }
Object.keys(user).forEach(k => console.log(k));
```

3 · Array — All Methods (grouped by mutation)

Ordered list. Knowing which methods MUTATE vs return a NEW array is a classic trap.

CRUD at a glance

```
// CREATE  []  Array.of(1,2)  Array.from('ab')  [...iterable]
// READ    arr[i]   arr.at(-1)   find()  filter()  includes()  indexOf()
// UPDATE  arr[i]=x  push/unshift  splice()  map() (new)  fill()
// DELETE  pop/shift  splice(i,1)  filter() (new, immutable)
```

Mutating methods (change the original array)

Method	What it does	Mutates
`push(...x)`	Add to **end**, returns new length	Yes
`pop()`	Remove & return **last** element	Yes
`unshift(...x)`	Add to **start**, returns new length	Yes
`shift()`	Remove & return **first** element	Yes
`splice(i,n,...x)`	Add/remove anywhere; returns removed items	Yes
`sort(cmp?)`	Sort in place (default = string order!)	Yes
`reverse()`	Reverse order in place	Yes
`fill(v,s?,e?)`	Fill range with a value	Yes

Non-mutating — return a new array / value

Method	What it does	Mutates
`slice(s?,e?)`	Shallow copy of a section	No
`concat(...)`	Merge arrays into a new one	No
`join(sep)`	Join into a string	No
`includes(v)`	`true`/`false` membership check	No
`indexOf(v)`	First index or `-1`	No
`at(i)`	Element by index (supports negative)	No
`flat(d?)`	Flatten nested arrays	No
`toSorted/toReversed`	ES2023 immutable versions of sort/reverse	No

Higher-order / iteration (callbacks)

Method	What it does	Returns
`forEach(fn)`	Run fn for each item (side effects)	undefined
`map(fn)`	Transform each item	new array
`filter(fn)`	Keep items where fn is truthy	new array
`reduce(fn,init)`	Fold to a single accumulated value	any
`find(fn)`	First matching **element**	element / undefined
`findIndex(fn)`	Index of first match	number / -1

Method	What it does	Returns
`some(fn)`	`true` if **any** item matches	boolean
`every(fn)`	`true` if **all** items match	boolean
`flatMap(fn)`	map then flatten one level	new array

```
const n = [1, 2, 3, 4];
n.map(x => x * 2);                // [2,4,6,8]
n.filter(x => x % 2 === 0);       // [2,4]
n.reduce((sum, x) => sum + x, 0); // 10
n.find(x => x > 2);               // 3
[1,[2,[3]]].flat(Infinity);       // [1,2,3]
[3,1,2].sort((a,b) => a - b);     // [1,2,3]  always pass a comparator!
```

4 · Set — unique values

Stores UNIQUE values of any type, in insertion order. O(1) membership checks.

CRUD + methods

```
// CREATE
const s = new Set([1, 2, 2, 3]);   // {1, 2, 3}  (dupes dropped)

// READ
s.has(2);          // true
s.size;            // 3   (property, not method!)

// UPDATE (add)
s.add(4);          // {1,2,3,4}  chainable, ignores duplicates

// DELETE
s.delete(1);       // true if removed
s.clear();         // empties the set

// ITERATE
for (const v of s) { /* ... */ }
s.forEach(v => console.log(v));

// FAVOURITE TRICK: dedupe an array
const unique = [...new Set([1,1,2,3,3])];   // [1,2,3]
```

Member	Does
`.add(v)`	Insert value (no duplicates), returns the Set (chainable)
`.has(v)`	`true`/`false` — fast membership test
`.delete(v)`	Remove a value, returns `true` if it existed
`.clear()`	Remove everything
`.size`	Count of elements (a property)

Set vs Array: use a Set when you need uniqueness and frequent `has()` checks — membership is O(1) vs an Array's O(n) `includes()`. Arrays keep duplicates and index access; Sets don't.

5 · Map — key→value with ANY key type

Like an object, but keys can be objects/functions, it keeps insertion order, and it's iterable.

CRUD + methods

```
// CREATE
const m = new Map([['id', 1], ['name', 'Suraj']]);

// READ
m.get('id');        // 1
m.has('name');      // true
m.size;             // 2   (property, not method!)

// UPDATE (set) — chainable
m.set('role', 'admin').set('id', 2);
const objKey = {};            // ANY type can be a key
m.set(objKey, 'value');

// DELETE
m.delete('role');   // true if removed
m.clear();          // empties the map

// ITERATE (insertion order)
for (const [k, v] of m) { console.log(k, v); }
m.forEach((v, k) => console.log(k, v));   // note: (value, key)
[...m.keys()]; [...m.values()]; [...m.entries()];
```

Member	Does
`.set(k,v)`	Insert/update a pair, returns the Map (chainable)
`.get(k)`	Value for key, or `undefined`
`.has(k)`	`true`/`false` if key exists
`.delete(k)`	Remove a key, returns `true` if it existed
`.clear()`	Remove all entries
`.size`	Number of entries (a property)
`.keys()/.values()/.entries()`	Iterators in insertion order

Map vs Object — the interview favourite

	Map	Object
Key types	Any value (objects, functions)	Only string / symbol
Order	Guaranteed insertion order	Mostly insertion (int keys sorted)
Size	`.size` property	`Object.keys(o).length`
Iterable	Directly (`for...of`)	Needs `Object.entries`
Performance	Better for frequent add/delete	Fine for static shapes
JSON	Not directly serializable	`JSON.stringify` works
Default keys	Clean (no prototype keys)	Inherits prototype keys

Rule of thumb: reach for a **Map** when keys are dynamic, non-string, or you add/remove a lot; use a plain **Object** for fixed, known-shape records and anything you'll `JSON.stringify`.

Last-minute recall: primitives copy by value, objects by reference · `const` object can mutate but not reassign · `map/filter/slice/concat` = new array · `push/pop/splice/sort/reverse` = mutate · `.size` for Set/Map, `.length` for Array · always pass a comparator to `sort()`.